

12 — Index Maintenance

"An index is not a fire-and-forget artifact — it requires regular care to stay healthy."

Table of Contents

1. [Learning Objectives](#)
 2. [Index Bloat Explained](#)
 3. [ASCII Diagram: Before and After Bloat](#)
 4. [Measuring Index Bloat](#)
 5. [VACUUM and Indexes](#)
 6. [REINDEX vs VACUUM](#)
 7. [REINDEX CONCURRENTLY](#)
 8. [Monitoring Index Health](#)
 9. [Identifying Unused Indexes](#)
 10. [Identifying Duplicate/Redundant Indexes](#)
 11. [Invalid Indexes](#)
 12. [Bloat from Dead Tuples](#)
 13. [pg_stat_user_indexes Reference](#)
 14. [Autovacuum Tuning for Index Health](#)
 15. [Index Maintenance Checklist](#)
 16. [Common Mistakes](#)
 17. [Best Practices](#)
 18. [Performance Considerations](#)
 19. [Interview Questions & Answers](#)
 20. [Exercises with Solutions](#)
 21. [Production Scenarios](#)
 22. [Cross-References](#)
-

Learning Objectives

After completing this section you will be able to:

- Explain how index bloat occurs and why it degrades performance
 - Measure index bloat using `pg_relation_size` and `pgstattuple`
 - Use `VACUUM` to reclaim dead index entries without rebuilding
 - Perform blocking and non-blocking `REINDEX` operations
 - Monitor index health using `pg_stat_user_indexes` and `pg_stat_user_tables`
 - Identify and remove unused, duplicate, or invalid indexes
 - Configure autovacuum appropriately for tables with heavy index usage
-

Index Bloat Explained

Index bloat is the accumulation of wasted space in index pages due to:

1. **Dead tuple markers:** when rows are updated or deleted, old index entries are marked dead but not immediately reclaimed
2. **Deleted pages:** when all entries on an index page are dead, the page is empty but not returned to the OS

3. **Page fragmentation:** after many deletes and inserts, pages may be only partially full

Why Bloat Occurs

```
INSERT row A → index entry for A created
UPDATE row A → creates new row A', marks old entry for A as dead
                  index now has both: [A (dead)] and [A' (live)]
DELETE row A' → marks A' as dead
                  index now has: [A (dead)] and [A' (dead)]

Without VACUUM:
  Over time, index fills with dead entries
  Pages that were 80% full become 80% dead + 20% live
  Query reads same number of pages but gets less useful data
```

Impact of Bloat

Bloat Level	Index Size vs. Optimal	Query Impact
0-20%	Normal	No impact
20-50%	Moderate	Slight slowdown (more pages to read)
50-80%	High	Noticeable slowdown
80%+	Severe	Significant performance degradation

ASCII Diagram: Before and After Bloat

HEALTHY INDEX PAGE (leaf, 8 KB):

[LIVE: k=1, ctid=(0,1)] [LIVE: k=2, ctid=(0,2)] [LIVE: k=3, ...] |

[LIVE: k=4, ctid=(0,4)] [LIVE: k=5, ctid=(1,1)] [LIVE: k=6, ...] |

... Fill = 85% |

Useful data: 85% of page |

BLOATED INDEX PAGE (same page after many updates/deletes):

[DEAD: k=1] [DEAD: k=2] [LIVE: k=3, ctid=(0,7)] [DEAD: k=4] |

[DEAD: k=5] [LIVE: k=6, ctid=(1,5)] [DEAD: k=7] [DEAD: k=8] |

... Fill = 85% |

LIVE data: only 15% of page! Dead entries: 70% |

A query scanning this bloated page reads 8 KB of data but only gets useful data from 15% of the page – 5.7× more I/O than necessary!

After VACUUM (dead entry removal, NOT full compaction):

[LIVE: k=3, ctid=(0,7)] [LIVE: k=6, ctid=(1,5)] |

[FREE SPACE]	Fill = 15%
LIVE data: 15%, free space: 85%	

Free space available for future inserts! But page count unchanged.

After REINDEX (full rebuild):

[LIVE: k=3, ctid=(0,7)] [LIVE: k=6, ctid=(1,5)]	
[and all other live entries packed in]	Fill = 85%
LIVE data: 85%, optimal packing	

New pages contain only live data, fully packed. File is smaller.

Measuring Index Bloat

Method 1: pgstattuple Extension (Most Accurate)

```
CREATE EXTENSION IF NOT EXISTS pgstattuple;

-- Measure a specific index
SELECT *
FROM pgstatindex('idx_orders_customer');

-- Output columns:
-- version:          pgstattuple version
-- tree_level:       B-tree height
-- index_size:       total index size in bytes
-- root_block_no:    root page number
-- internal_pages:   number of internal pages
-- leaf_pages:       number of leaf pages
-- empty_pages:      number of completely empty pages
-- deleted_pages:    number of deleted pages
-- avg_leaf_density: average fill factor of leaf pages (%)
-- leaf_fragmentation: % of leaf pages not contiguous

-- Interpreting results:
-- avg_leaf_density < 50%: significant bloat, consider REINDEX
-- leaf_fragmentation > 50%: high fragmentation, consider REINDEX
-- deleted_pages > 10% of leaf_pages: pages to be reclaimed by VACUUM
```

Method 2: Estimation via pg_relation_size

```
-- Quick size check
SELECT
    indexname,
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size,
    pg_size_pretty(pg_table_size(indrelid)) AS table_size,
    ROUND(100.0 * pg_relation_size(indexrelid) / pg_table_size(indrelid), 1) AS
idx_pct_of_table
```

```

FROM pg_stat_user_indexes
JOIN pg_index USING (indexrelid)
WHERE relname = 'orders'
ORDER BY pg_relation_size(indexrelid) DESC;

-- Red flags: index is more than 50% of table size for a simple B-tree

```

Method 3: bloat estimation query (no extension needed)

```

-- Approximate bloat estimation based on pg_stats
SELECT
    schemaname,
    tablename,
    indexname,
    bs * (relpages::bigint) AS real_size,
    bs * (relpages::bigint - est_pages) AS bloat_size,
    ROUND(100.0 * (relpages::bigint - est_pages) / relpages, 1) AS bloat_ratio
FROM (
    SELECT
        schemaname,
        tablename,
        indexname,
        bs,
        relpages,
        -- estimated ideal pages based on tuple count and avg key size
        CEIL(reltuples / ((bs - 24) / (nulldatahdrwidth + 2 + datawidth +
6.0))))::BIGINT AS est_pages
    FROM (
        SELECT
            schemaname,
            tablename,
            indexname,
            current_setting('block_size')::INTEGER AS bs,
            pg_class.relpages,
            pg_class.reltuples,
            -- approximate per-entry size (simplified)
            8 AS nulldatahdrwidth,
            24 AS datawidth
        FROM pg_indexes
        JOIN pg_class ON pg_indexes.indexname = pg_class.relname
        WHERE schemaname NOT IN ('pg_catalog', 'information_schema')
    ) raw
) est
WHERE relpages > 0
ORDER BY bloat_size DESC
LIMIT 20;

```

VACUUM and Indexes

VACUUM is the primary maintenance tool that removes dead index entries without a full rebuild.

What VACUUM Does to Indexes

- 1. Scans heap pages for dead tuples
- 2. For each dead tuple found, removes the corresponding index entry
- 3. Marks index pages that become empty as "deleted" (available for reuse)
- 4. Updates the free space map

What VACUUM Does NOT Do

- Does NOT repack pages (doesn't compact remaining entries)
- Does NOT reclaim the underlying file pages to the OS
- Does NOT sort index entries
- Does NOT reduce the physical file size (that requires VACUUM FULL or REINDEX)

```
-- Standard VACUUM (non-blocking, cleans dead entries)
VACUUM orders;

-- VACUUM with analysis (update planner statistics too)
VACUUM ANALYZE orders;

-- VACUUM FULL (rewrites the table AND all indexes – EXCLUSIVE LOCK!)
-- Only use if desperate for space; blocks all access during operation
VACUUM FULL orders;
-- After VACUUM FULL, physical file shrinks. Equivalent to dump+restore for file
size.
```

REINDEX vs VACUUM

Operation	What it Does	Locking	When to Use
VACUUM	Removes dead entries, frees page space for reuse	Minimal	Routine maintenance, light bloat
REINDEX	Completely rebuilds index from scratch	ShareLock (blocks writes)	Severe bloat, corruption, optimization
REINDEX CONCURRENTLY	Rebuilds index, minimal locking	Brief locks at end	Production tables needing rebuild

REINDEX Options

```
-- Rebuild a single index (BLOCKS WRITES during rebuild!)
REINDEX INDEX idx_orders_customer;

-- Rebuild all indexes on a table (BLOCKS WRITES!)
REINDEX TABLE orders;

-- Rebuild all indexes in a schema
REINDEX SCHEMA public;

-- Rebuild all system indexes in the database
```

```
REINDEX SYSTEM mydb;

-- Rebuild entire database
REINDEX DATABASE mydb;
```

REINDEX CONCURRENTLY

Available since PostgreSQL 12. Rebuilds the index without blocking reads OR writes for the duration of the build.

How REINDEX CONCURRENTLY Works

```
Phase 1: Create a new index with a temporary name (concurrent build)
        → Table is fully accessible during this phase
Phase 2: Wait for all transactions that started before phase 1 to finish
Phase 3: Briefly acquire a ShareUpdateExclusiveLock to swap old and new indexes
Phase 4: Drop the old index
```

```
-- Non-blocking rebuild (PostgreSQL 12+)
REINDEX INDEX CONCURRENTLY idx_orders_customer;

-- Non-blocking rebuild of all indexes on a table
REINDEX TABLE CONCURRENTLY orders;

-- Non-blocking rebuild of entire schema
REINDEX SCHEMA CONCURRENTLY public;
```

REINDEX CONCURRENTLY Caveats

1. **Takes longer** than regular REINDEX (multiple passes)
2. **Cannot be run inside a transaction block**
3. **Leaves an INVALID index** if it fails — must be dropped manually
4. **Needs extra disk space** — old and new index coexist during build
5. **Cannot rebuild** system catalogs, indexes on partitioned tables (parent level), or unique indexes without temporarily relaxing uniqueness

Handling Failed REINDEX CONCURRENTLY

```
-- If REINDEX CONCURRENTLY fails, an INVALID index may be left
SELECT indexname, pg_get_indexdef(indexrelid) AS definition
FROM pg_indexes
JOIN pg_class ON pg_indexes.indexname = pg_class.relname
WHERE pg_class.relname NOT IN (
    SELECT indexname FROM pg_stat_user_indexes
    WHERE idx_scan >= 0 -- just to join
)
-- Better check:
SELECT indexname FROM pg_indexes WHERE indexdef ILIKE '%invalid%';
```

```
-- Or query pg_index directly:
SELECT indexrelid::regclass AS index_name
FROM pg_index
WHERE NOT indisvalid;

-- Drop the invalid index and retry:
DROP INDEX CONCURRENTLY idx_orders_customer_new; -- temp name from failed build
REINDEX INDEX CONCURRENTLY idx_orders_customer;
```

Monitoring Index Health

Master Health Query

```
SELECT
    schemaname AS schema,
    relname AS table,
    indexrelname AS index,
    idx_scan AS scans,
    idx_tup_read AS tuples_read,
    idx_tup_fetch AS tuples_fetched,
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size,
    -- Calculate bloat proxy
    CASE
        WHEN idx_scan = 0 THEN 'UNUSED'
        WHEN idx_scan < 100 THEN 'RARELY_USED'
        ELSE 'ACTIVE'
    END AS usage_status,
    -- Check if invalid
    CASE WHEN NOT indisvalid THEN 'INVALID!' ELSE 'valid' END AS validity
FROM pg_stat_user_indexes
JOIN pg_index USING (indexrelid)
WHERE schemaname NOT IN ('pg_catalog', 'information_schema')
ORDER BY pg_relation_size(indexrelid) DESC;
```

Statistics Reset

pg_stat_user_indexes counters are reset when `pg_stat_reset()` is called or the server restarts.

```
-- When were statistics last reset?
SELECT stats_reset FROM pg_stat_bgwriter;

-- Reset statistics (careful! loses all usage history)
SELECT pg_stat_reset();
```

Identifying Unused Indexes

An "unused" index has `idx_scan = 0` or very low scan count since last stats reset.

```
-- Find unused indexes (with size information)
SELECT
    schemaname || '.' || relname AS table,
    indexrelname AS index_name,
    pg_size_pretty(pg_relation_size(indexrelid)) AS wasted_size,
    idx_scan AS times_used,
    pg_get_indexdef(indexrelid) AS definition
FROM pg_stat_user_indexes
JOIN pg_index USING (indexrelid)
WHERE idx_scan = 0
    AND NOT indisunique          -- don't drop unique indexes (used for constraints)
    AND NOT indisprimary         -- don't drop PKs
    AND schemaname NOT IN ('pg_catalog', 'information_schema')
ORDER BY pg_relation_size(indexrelid) DESC;
```

Before dropping any index:

1. Verify the statistics period is representative (at least 30 days, including peak load)
2. Check if the index enforces a constraint (unique, FK)
3. Test the impact of dropping on query plans: `SET enable_indexscan = off` and re-run queries

```
-- "Soft drop" – rename first, monitor for issues, then drop
ALTER INDEX idx_orders_rarely_used RENAME TO
_unused_idx_orders_rarely_used_20240615;
-- After 30 days with no issues:
DROP INDEX CONCURRENTLY _unused_idx_orders_rarely_used_20240615;
```

Identifying Duplicate/Redundant Indexes

```
-- Find exact duplicates (same columns, same definition)
SELECT
    a.indexname AS index1,
    b.indexname AS index2,
    a.tablename,
    a.indexdef
FROM pg_indexes a
JOIN pg_indexes b
    ON a.tablename = b.tablename
    AND a.indexname < b.indexname
    AND a.indexdef = b.indexdef;

-- Find redundant indexes (a is a prefix of b)
-- e.g., index on (a) when (a, b) also exists
SELECT
    a.indexname AS shorter_index,
    b.indexname AS longer_index,
    a.tablename
FROM pg_indexes a
JOIN pg_indexes b
```



```

    ON a.tablename = b.tablename
    AND a.indexname != b.indexname
    AND b.indexdef LIKE a.indexdef || '%' -- rough check
WHERE length(a.indexdef) < length(b.indexdef);

-- More precise: using pg_index columns
SELECT
    short_idx.indexrelid::regclass AS short_index,
    long_idx.indexrelid::regclass AS long_index,
    short_idx.indrelid::regclass AS table_name
FROM pg_index short_idx
JOIN pg_index long_idx
    ON short_idx.indrelid = long_idx.indrelid
    AND short_idx.indexrelid != long_idx.indexrelid
    -- Short index columns are prefix of long index columns
    AND short_idx.indkey::text = left(long_idx.indkey::text,
length(short_idx.indkey::text))
    AND NOT short_idx.indisunique -- keep unique indexes even if "redundant"
WHERE NOT short_idx.indisprimary;

```

Invalid Indexes

An invalid index is one that failed to complete building (from a failed `CREATE INDEX CONCURRENTLY`) or is being rebuilt.

```

-- Find invalid indexes
SELECT
    schemaname,
    tablename,
    indexname,
    pg_size_pretty(pg_relation_size(indexrelid)) AS size
FROM pg_indexes
JOIN pg_class ON pg_indexes.indexname = pg_class.relname
JOIN pg_index ON pg_class.oid = pg_index.indexrelid
WHERE NOT pg_index.indisvalid
    AND schemaname NOT IN ('pg_catalog');

-- An invalid index is not used by the planner but still causes write overhead!
-- Drop it immediately and re-create:
DROP INDEX CONCURRENTLY invalid_index_name;
REINDEX INDEX CONCURRENTLY idx_real_index_name;
-- or:
CREATE INDEX CONCURRENTLY idx_real_index_name ON table(column);

```

pg_stat_user_indexes Reference

Column	Description
relid	OID of the indexed table

indexrelid	OID of the index
schemaname	Schema name
relname	Table name
indexrelname	Index name
idx_scan	Number of index scans
idx_tup_read	Number of index entries returned by scans
idx_tup_fetch	Number of live heap tuples fetched by index scans

`idx_tup_read` vs `idx_tup_fetch` : the difference is Bitmap Index Scans, which read index entries but don't immediately fetch heap tuples.

Autovacuum Tuning for Index Health

Default Settings

```
-- Global autovacuum settings
SHOW autovacuum_vacuum_scale_factor; -- default 0.2 (vacuum after 20% rows changed)
SHOW autovacuum_vacuum_threshold;    -- default 50 (vacuum after 50 dead rows min)
SHOW autovacuum_analyze_scale_factor; -- default 0.1
SHOW autovacuum_analyze_threshold;   -- default 50
```

Tuning for High-Churn Tables

```
-- For a table with 10M rows and frequent updates/deletes:
-- Default: vacuum triggers at 20% = 2,000,000 dead tuples before vacuuming
-- That's too much bloat! Tune per-table:

ALTER TABLE orders SET (
    autovacuum_vacuum_scale_factor = 0.05, -- vacuum after 5% dead rows
    autovacuum_vacuum_threshold = 1000,    -- vacuum after 1000 dead rows min
    autovacuum_analyze_scale_factor = 0.02 -- analyze more frequently
);

-- For very hot tables (e.g., session or queue tables with millions of rows):
ALTER TABLE user_sessions SET (
    autovacuum_vacuum_scale_factor = 0.01,
    autovacuum_vacuum_threshold = 100,
    autovacuum_vacuum_cost_delay = 0 -- don't throttle autovacuum for this table
);
```

Monitoring Autovacuum Activity

```
SELECT
    relname,
    last_vacuum,
    last_autovacuum,
    last_analyze,
    last_autoanalyze,
    n_dead_tup,
    n_live_tup,
    ROUND(100.0 * n_dead_tup / NULLIF(n_live_tup + n_dead_tup, 0), 1) AS dead_pct,
    autovacuum_count
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC
LIMIT 20;
```

Index Maintenance Checklist

Weekly

- ☐ Check `pg_stat_user_indexes` for `idx_scan = 0` indexes
- ☐ Check `pg_stat_user_tables` for tables with high `n_dead_tup`
- ☐ Verify autovacuum is running: `SELECT * FROM pg_stat_bgwriter`

Monthly

- ☐ Run `pgstattuple` on large critical indexes; check `avg_leaf_density`
- ☐ Identify and drop unused indexes (after verifying with team)
- ☐ Check for invalid indexes
- ☐ Review duplicate/redundant indexes

Quarterly

- ☐ `REINDEX CONCURRENTLY` any indexes with `avg_leaf_density < 50%`
- ☐ Review index strategy for the most critical 10 tables
- ☐ Update planner statistics: `ANALYZE` on recently-changed tables

After Bulk Operations

- ☐ After bulk INSERT/COPY: `ANALYZE table_name`
- ☐ After bulk UPDATE/DELETE: `VACUUM ANALYZE table_name`
- ☐ After major schema changes: `REINDEX TABLE CONCURRENTLY table_name`

Common Mistakes

1. Running REINDEX (blocking) on production tables

```
-- NEVER do this on a production table without a maintenance window:
REINDEX TABLE orders; -- exclusive lock, blocks all queries!
```

```
-- Always use:  
REINDEX TABLE CONCURRENTLY orders;
```

2. Not monitoring for invalid indexes after CONCURRENT operations

Failed `CREATE INDEX CONCURRENTLY` leaves invalid indexes that consume write overhead but provide no benefit. Always check:

```
SELECT * FROM pg_index WHERE NOT indisvalid;
```

3. Assuming VACUUM reclaims disk space

`VACUUM` does NOT reduce the physical size of the index file. It only marks pages as reusable internally. To actually shrink the file, you need `VACUUM FULL` or `REINDEX`.

4. Not adjusting autovacuum for high-churn tables

Default `autovacuum_vacuum_scale_factor = 0.2` means a 10M-row table needs 2M dead tuples before autovacuum triggers. Tune this aggressively for tables with heavy UPDATE/DELETE.

5. Dropping indexes without verifying constraint enforcement

Some indexes enforce UNIQUE constraints or are used by FK checks. Dropping them removes the constraint silently (or causes an error for FK-related indexes).

```
-- Check if index enforces constraint before dropping:  
SELECT conname, contype FROM pg_constraint  
WHERE conindid = 'idx_to_drop'::regclass::oid;
```

Best Practices

1. Use `REINDEX CONCURRENTLY` for all production index rebuilds.
2. Monitor `pg_stat_user_indexes` weekly — set up automated alerts for unused indexes.
3. Tune autovacuum per-table for high-churn tables to prevent bloat accumulation.
4. After any bulk operation, run `VACUUM ANALYZE` on affected tables.
5. Use `pg_stat_reset()` at the start of a monitoring period, wait 30 days, then review usage statistics for cleanup decisions.
6. Never drop an index that enforces a constraint without a migration plan.
7. Test index drops in a staging environment with production-like query patterns before applying to production.
8. Schedule `REINDEX CONCURRENTLY` for indexes showing `avg_leaf_density < 50%` in `pgstattuple` reports.

Performance Considerations

Cost of Bloat on Queries

A bloated index with 70% dead entries needs to read 3.3× more pages to find the same amount of live data. For an index used 10,000 times/second, this directly translates to 3.3× more shared_buffers usage and I/O.

REINDEX vs Table Partition Rotation

For very large tables, consider partition rotation as an alternative to REINDEX:

- Keep data in monthly partitions
- Drop old partitions (no VACUUM needed for dropped partitions)
- Attach new partitions for new data (no index rebuild needed)

Parallel VACUUM

PostgreSQL 13+ supports parallel index vacuuming:

```
-- Vacuum using 4 workers
VACUUM (PARALLEL 4) large_table;
```

Interview Questions & Answers

Q1. What is index bloat and how does it affect query performance?

A: Index bloat occurs when dead tuples (from UPDATES and DELETES) accumulate in index pages but aren't reclaimed. Each dead entry still occupies space in the leaf page. A bloated index has pages that appear full but contain mostly dead entries. Queries must read more pages to find the same number of live entries, increasing I/O and reducing cache efficiency. For example, an index with 80% dead entries needs to read 5× more pages than an optimal index for the same result set. Bloat also increases the physical index file size, using more disk and buffer cache.

Q2. What is the difference between VACUUM and REINDEX for index maintenance?

A: VACUUM removes dead entries from index pages and marks pages as available for reuse, but does NOT compact remaining entries or shrink the file. Pages remain at the same physical size. REINDEX completely rebuilds the index from scratch: it creates a new, compactly-packed index file and drops the old one. REINDEX eliminates all fragmentation and reduces the file size, but requires exclusive locks (blocking writes) unless CONCURRENTLY is used. VACUUM is lightweight and runs automatically; REINDEX is a heavier operation done when bloat is severe.

Q3. How does REINDEX CONCURRENTLY work and what are its limitations?

A: REINDEX CONCURRENTLY (PG12+) works in multiple phases: first, it builds a new index while allowing concurrent reads and writes (the existing index is still used). Then, it briefly acquires a lock to swap the old index with the new one. Finally, it drops the old index. Limitations: (1) Cannot be run inside a transaction. (2) Takes longer than regular REINDEX (multiple passes). (3) Requires extra disk space while both indexes coexist. (4) If it fails, it leaves an INVALID index that must be manually dropped. (5) Cannot rebuild some special indexes (partitioned table parent indexes, system catalogs).

Q4. How do you identify and safely remove unused indexes?

A: Query `pg_stat_user_indexes` filtering for `idx_scan = 0`. Before removing, ensure statistics were collected over a representative period (30+ days including peak load). Check the index is not enforcing a UNIQUE constraint or FK check. Consider "soft dropping" — renaming the index with a prefix like `_unused_` and monitoring for 30 days before dropping. Always use `DROP INDEX CONCURRENTLY` to avoid locking the table.

Q5. Why might a REINDEX CONCURRENTLY leave an invalid index, and how do you handle it?

A: If `REINDEX CONCURRENTLY` is interrupted (network failure, server crash, cancellation), it may leave a partially-built index marked as `INVALID` in `pg_index`. An invalid index receives writes (increasing write overhead) but is never used by the planner — worst of both worlds. To handle it: find invalid indexes with `SELECT * FROM pg_index WHERE NOT indisvalid`, drop the invalid one with `DROP INDEX CONCURRENTLY`, then re-run `REINDEX CONCURRENTLY` or `CREATE INDEX CONCURRENTLY` to rebuild it.

Exercises with Solutions

Exercise 1

Write a complete monitoring query that identifies: (a) unused indexes, (b) very large indexes relative to their table, and (c) invalid indexes.

Solution:

```
SELECT
    schemaname AS schema,
    relname AS table_name,
    indexrelname AS index_name,
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size,
    pg_size_pretty(pg_relation_size(indrelid)) AS table_size,
    ROUND(100.0 * pg_relation_size(indexrelid) /
        NULLIF(pg_relation_size(indrelid), 0), 1) AS idx_pct,
    idx_scan AS scan_count,
    indisunique AS is_unique,
    indisprimary AS is_primary,
    NOT indisvalid AS is_invalid,
    CASE
        WHEN NOT indisvalid THEN 'INVALID - drop and recreate!'
        WHEN idx_scan = 0 AND NOT indisunique AND NOT indisprimary THEN 'UNUSED -
review for drop'
        WHEN pg_relation_size(indexrelid) > pg_relation_size(indrelid) * 0.5 THEN
'LARGE - check for bloat'
        ELSE 'OK'
    END AS recommendation
FROM pg_stat_user_indexes
JOIN pg_index USING (indexrelid)
WHERE schemaname NOT IN ('pg_catalog', 'information_schema')
ORDER BY
    NOT indisvalid DESC,      -- invalid first
    idx_scan ASC,             -- unused next
    pg_relation_size(indexrelid) DESC;
```

Production Scenarios

Scenario 1: Index Bloat Crisis

A `user_events` table (50M rows) has an index that was once 500 MB but has grown to 3 GB after 6 months of high UPDATE/DELETE activity. Query performance has degraded 5×.

```
-- Step 1: Verify bloat with pgstattuple
CREATE EXTENSION IF NOT EXISTS pgstattuple;
SELECT * FROM pgstatindex('idx_user_events_user_id');
-- avg_leaf_density: 23% (expected: 70-90%)
-- leaf_fragmentation: 78%

-- Step 2: Rebuild concurrently
REINDEX INDEX CONCURRENTLY idx_user_events_user_id;
-- Duration: ~45 minutes for 50M rows (non-blocking)

-- Step 3: Verify improvement
SELECT * FROM pgstatindex('idx_user_events_user_id');
-- avg_leaf_density: 85% ← restored to optimal
-- New index size: 420 MB (down from 3 GB)

-- Step 4: Tune autovacuum to prevent recurrence
ALTER TABLE user_events SET (
    autovacuum_vacuum_scale_factor = 0.05,
    autovacuum_vacuum_threshold = 10000
);
```

Cross-References

- [01 index fundamentals.md](#) — Index basics
- [02 btree indexes.md](#) — B-tree page structure
- [08 partial indexes.md](#) — Partial indexes for reducing bloat
- [../08 Query Optimization/09 slow query analysis.md](#) — Identifying slow queries from bloated indexes
- [../05 PostgreSQL Core/autovacuum.md](#) — Autovacuum configuration